



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Développement web back-end

Les fonctions en PHP

Sommaire

1	Introduction aux fonctions.....	2
1.1	Les fonctions internes.....	2
1.2	Les fonctions définies par l'utilisateur.....	2
1.3	Paramètre et arguments des fonctions.....	2
2	Contrôler le passage des arguments.....	3
2.1	Passer des arguments par référence.....	3
2.2	Définir des valeurs par défaut pour les paramètres	5
2.3	Créer une fonction avec un nombre de paramètres indéterminé.....	5
2.4	PHP, un langage au typage faible	6
2.4.1	Le typage des arguments	6
3	Contrôler les valeurs de retour d'une fonction	7
3.1	Avantages et spécificités de l'instruction return.....	7
3.2	Déclaration des types de valeur de retour	8
4	La portée des variables en PHP.....	8
5	Constantes et constantes magiques en PHP	8
5.1	Définition	8
5.2	Créer ou définir une constante.....	8
5.3	Les constantes prédéfinies et les constantes magiques en PHP.....	9

1 Introduction aux fonctions

Les fonctions sont l'un des outils les plus puissants et les plus pratiques du PHP et vous devez absolument savoir les manipuler.

Une fonction correspond à une série cohérente d'instructions qui ont été créées pour effectuer une tâche précise. Pour exécuter le code contenu dans une fonction, il va falloir appeler la fonction.

1.1 Les fonctions internes

Nous avons déjà croisé des fonctions dans les exemples précédents avec notamment la fonction `gettype()` dont le rôle est de retourner le type d'une variable.

La fonction `gettype()` est une fonction dite interne de PHP car elle est livrée avec le langage.

Il existe un très grand nombre de fonctions internes, comme par exemple les fonctions sur les [chaines de caractères](#).

1.2 Les fonctions définies par l'utilisateur

En plus des nombreuses fonctions internes de PHP, nous pouvons définir nos propres fonctions pour répondre à des besoins spécifiques.

Contrairement aux variables, le nom des fonctions est insensible à la casse, par contre il devra suivre les mêmes règles quant au nommage. A savoir :

- Pas de caractères spéciaux
- Débuter par une lettre ou un _
- Pas d'espace
- Pas de nom réservé

Voici à quoi peut ressembler la déclaration d'une variable classique :

```
<?php
function bonjour() {
    echo 'Bonjour à tous';
}
?>
```

Et il faut maintenant appeler la fonction `bonjour()`.

```
<?php
function bonjour() {
    echo 'Bonjour à tous <br>';
}
bonjour();
bonjour();
bonjour();
?>
```

1.3 Paramètre et arguments des fonctions

Quelles différences entre un paramètre et un argument ? On parlera de paramètre lors de la définition d'une fonction. Un paramètre sert à indiquer qu'une fonction va avoir besoin d'un

argument pour fonctionner mais ne correspond pas à une valeur effective : ce n'est qu'un prête nom. Un argument en revanche correspond à la valeur effective passée à une fonction.

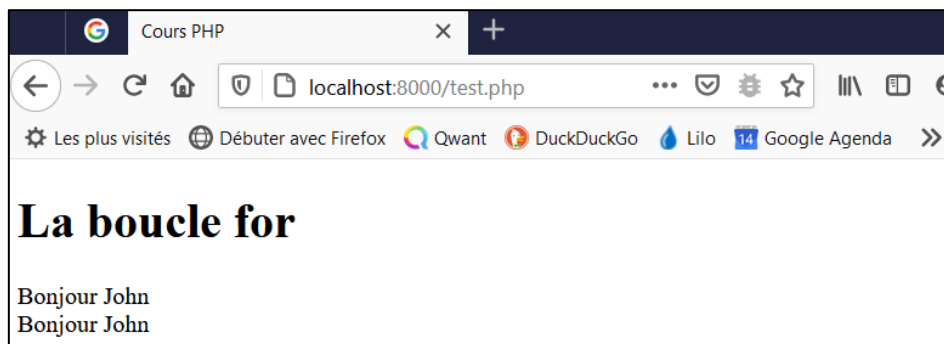
Parfois, les fonctions vont avoir besoin d'informations externes pour effectuer leur tâche. Par exemple la fonction `gettype()` ne fait rien si nous ne lui donnons pas la variable dont nous souhaitons connaître le type. Nous devons donc utiliser cette fonction de cette manière `gettype($a)` pour connaître le type de `$a`.

De la même manière nous pouvons modifier notre fonction `bonjour()` pour qu'elle dise bonjour à quelqu'un.

```
<?php
    $nom= "John";

    function bonjour($n) {
        echo 'Bonjour ' . $n . '<br>';
    }

    bonjour($nom);
    bonjour("John");
?>
```



Le nombre de paramètres importe peu :

```
<?php
    function addition($a, $b) {
        echo $a + $b;
    }

    addition(3, 12);
?>
```

2 Contrôler le passage des arguments

2.1 Passer des arguments par référence

Jusqu'à maintenant, nous avons passé nos arguments par valeur ce qui correspond au passage par défaut de PHP.

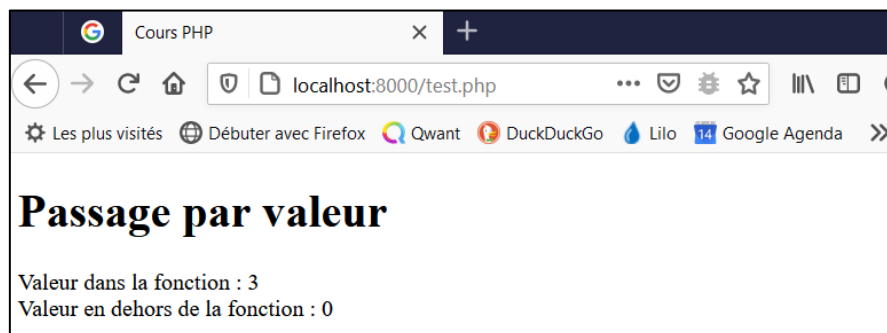
Lorsqu'on passe une variable comme argument par valeur à une fonction, le fait de modifier la valeur de la variable à l'intérieur de la fonction ne va pas modifier sa valeur à l'extérieur de la fonction.

Regardons cet exemple :

```
<?php
    $x = 0;

    function plus3($p) {
        $p += 3;
        echo 'Valeur dans la fonction : ' . $p;
    }

    plus3($x);
    echo '<br>Valeur en dehors de la fonction : ' . $x;
?>
```



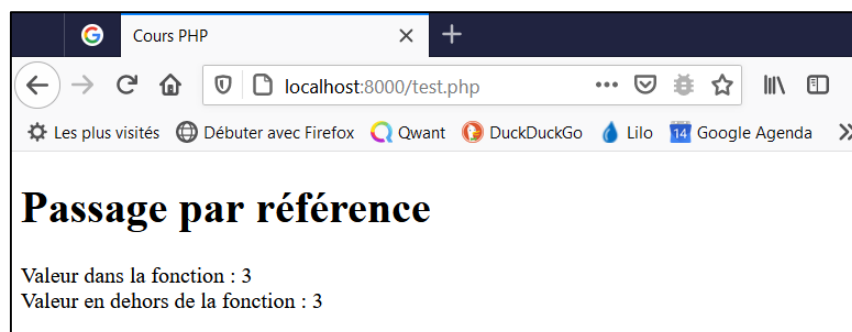
Par défaut, une fonction ne modifie pas son environnement extérieur.

Essayons maintenant de dire à notre fonction qu'elle recevra un [paramètre par référence](#) en la préfixant de `&`.

```
<?php
    $x = 0;

    function plus3(&$p) {
        $p += 3;
        echo 'Valeur dans la fonction : ' . $p;
    }

    plus3($x);
    echo '<br>Valeur en dehors de la fonction : ' . $x;
?>
```



Maintenant notre fonction modifie son environnement extérieur. Soyez prudent avec cette pratique.

2.2 Définir des valeurs par défaut pour les paramètres

Nous pouvons définir des valeurs par défaut pour les paramètres des fonctions.

```
<?php
function bonjour($n = 'John') {
    echo "Bonjour $n <br>";
}

bonjour("Dora");
bonjour();
?>
```



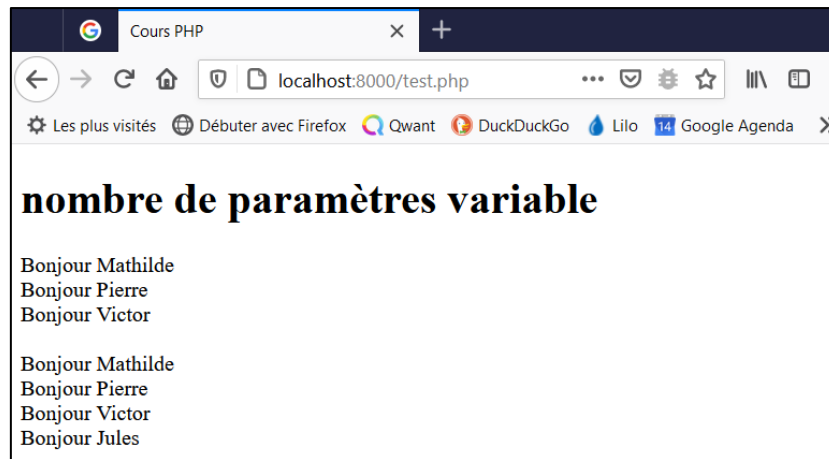
Si l'argument est passé, il est pris en compte, sinon c'est l'argument par défaut qui est utilisé par la fonction.

2.3 Créer une fonction avec un nombre de paramètres indéterminé

Nous pouvons créer des fonctions dont le nombre de paramètres ne peut pas être connu à l'avance. PHP prend en compte cette possibilité.

```
<?php
function bonjour(...$prenoms){
    foreach($prenoms as $p){
        echo 'Bonjour ' . $p . ' <br>';
    }
}

bonjour('Mathilde', 'Pierre', 'Victor');
echo ' <br>';
bonjour('Mathilde', 'Pierre', 'Victor', 'Jules');
?>
```



2.4 PHP, un langage au typage faible

Du fait que PHP est un langage qui ne possède pas de typage fort, il est possible de passer des arguments qui n'ont aucun sens à une fonction. Nous pourrions passer des chaînes de caractères à notre précédente fonction `addition()`.

2.4.1 Le typage des arguments

Depuis sa dernière version majeure (PHP7), le PHP nous permet de préciser les types des argument attendus par une fonction.

Examinons le code suivant et son retour :

```
<?php
function addition($a, $b) {
    echo $a + $b;
}

addition(5, 7);
echo '<br>';
addition("toto", "tata");
echo '<br><br>';

function addition2(float $a, float $b) {
    echo $a + $b;
}

addition2(5, 7);
echo '<br>';
addition2("toto", "tata");
echo '<br><br>';
?>
```



Dans notre première fonction `addition()` les paramètres ne sont pas typés. Le premier appel est conforme et PHP effectue correctement l'addition. Lors du second appel, les arguments n'ont aucun sens et PHP les convertis en `0`.

Dans notre seconde fonction `addition2()`, les paramètres sont typés `float` et lors de l'appel de la fonction avec des chaînes de caractère, PHP n'évalue pas la fonction, d'où l'absence de `0`.

3 Contrôler les valeurs de retour d'une fonction

3.1 Avantages et spécificités de l'instruction `return`

Jusqu'à présent, le seul moyen que nous avons d'avoir accès au résultat d'une fonction qu'on avait créée était d'afficher ce résultat en utilisant un `echo` dans la définition de la fonction lors de sa création.

Le problème de cette méthode est que nous n'avons aucun contrôle sur le résultat de la fonction : dès que l'on appelle la fonction en question, le `echo` à l'intérieur est exécuté et le résultat est immédiatement affiché.

La structure de contrôle `return` va nous permettre de demander à une fonction de retourner un résultat qu'on va ensuite pouvoir stocker dans une variable ou autre pour le manipuler.

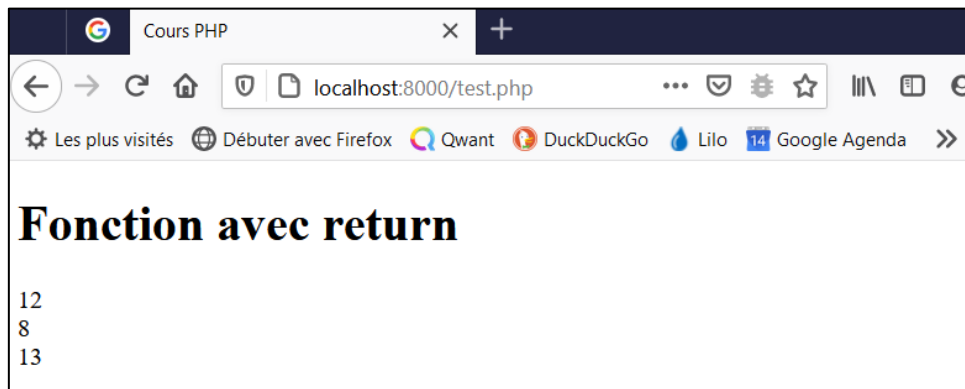
Attention l'instruction `return` va terminer l'exécution d'une fonction. Une instruction placée après un `return` ne sera jamais exécuté.

Examinons ce code et son résultat :

```
<?php
function addition($a, $b) {
    return $a + $b;
    echo 'Vous ne me verrez jamais';
}
$resultat = addition(3, 5);

echo addition(5, 7);
echo '<br>';
echo $resultat;
echo '<br>';
echo addition($resultat, 5);

?>
```

3.2 Déclaration des types de valeur de retour

Comme pour la déclaration des types des arguments, PHP7 permet de donner le type de valeur de retour.

La notation se fait ainsi :

```
<?php
    function addition($a, $b): float {
        return $a + $b;
        echo 'Vous ne me verrez jamais';
    }
?>
```

4 La portée des variables en PHP

En PHP, il est possible de déclarer des variables n'importe où dans notre script : au début, à la fin, à l'intérieur d'une boucle, au sein d'une fonction, etc.

L'idée principale à retenir ici est que l'endroit dans le script où on déclare une variable va déterminer l'endroit où la variable va être accessible c'est-à-dire utilisable. La « portée » d'une variable désigne justement la partie du script où la variable va être accessible.

5 Constantes et constantes magiques en PHP

5.1 Définition

Les constantes sont des conteneurs qui vont servir à stocker une valeur. Le contenu d'une constante ne peut pas être modifié (sauf pour les constantes magiques).

Par convention, les constantes sont écrites en majuscule.

5.2 Créer ou définir une constante

Pour définir une constante en PHP, nous allons pouvoir utiliser la fonction `define()` ou le mot clef `const`.

Pour créer une constante avec `define()`, nous devons passer en arguments le nom de la constante et la valeur `define('DIX', 10);`.

Pour créer une constante avec `const`, nous utiliserons la même syntaxe que pour les variables : `const DIX = 10;`.

```
<?php
    define('DIX', 10);
    const TRENTE = 30;
    echo 'La constante DIX contient: ' . DIX;
    echo '<br>';
    echo 'La constante TRENTE contient: ' . TRENTE;
?>
```



5.3 Les constantes prédéfinies et les constantes magiques en PHP

Le PHP nous fournit également un certain nombre de constantes prédéfinies qui vont généralement nous donner des informations de type « meta » comme la version de PHP actuellement utilisée, le plus petit entier supporté par PHP, des constantes de rapport d'erreur etc.

Parmi ces constantes prédéfinies, il y en a neuf qui vont retenir notre attention et qu'on appelle des constantes « magiques ». Ces constantes se distinguent des autres puisque leur valeur va changer en fonction de l'endroit dans le script où elles vont être utilisées.

Les constantes magiques sont reconnaissables par le fait que leur nom commence par deux underscores et se termine de la même manière (excepté pour une). Ce sont les suivantes :

Découvrir :

- Les constantes [magiques](#)
- Les constantes [prédéfinies](#)

Les constantes magiques vont notamment être utiles pour récupérer des adresses de fichiers et les utiliser dans des fichiers de configuration ou lors d'appels à nos bases de données SQL ou encore pour effectuer des actions de débogage.